

NICE ASSISTANT - Complete Project Dossier

Comprehensive Idea, PRS, Implementation Plans, Data Flow Diagrams, and System Design

Date: March 7, 2026

Project Root: F:/Hackathon Project

Primary Target: Android (offline-default), with desktop web support

Document Scope

- Whole project idea and product vision.
- Project Requirement Specifications (functional + non-functional).
- Original build plan and hardening implementation plan.
- Architecture and design of all major modules.
- All key data-flow diagrams and execution sequences.
- Testing strategy, release process, and stability/risk forecast.

1) Full Project Idea and Vision

Nice Assistant is designed as a privacy-first personal assistant that runs primarily on-device. The core idea is to replace cloud-dependent assistant behavior with deterministic local execution, local search and OCR, and optional online features only when the user explicitly enables them for the current session.

The assistant must understand natural language commands, operate device features, and answer questions from the user's local files.

The architecture prioritizes edge inference, low-latency action handling, and Android compatibility (Android 11+ scoped storage behavior included).

Offline behavior is the baseline: no hidden cloud fallback for core tasks.

Design goal: smooth usage on low-end hardware (4GB RAM class devices) without ANR or UI lockups.

World-Class Experience Targets

- Fast and truthful responses: avoid simulated outputs and explicitly state unavailable features.
- Content-grounded answers from files, with confidence and evidence.
- Reliable voice stack: wake word, STT/TTS, microphone arbitration.
- Action automation: deterministic command execution and local trigger-action rules.
- High reliability under storage permission edge cases and background/foreground transitions.

2) Project Requirement Specifications (PRS)

2.1 Stakeholders and Target Users

- Privacy-focused users who do not want cloud data processing.
- Users in low/no internet connectivity environments.
- Students and professionals requiring deep local document search.
- Power users who want on-device automations and voice control.

2.2 Functional Requirements

ID	Requirement	Status
FR-01	Offline-first command parser and deterministic intent routes	Implemented
FR-02	Deep file indexing across Android-accessible storage roots	Implemented
FR-03	Content extraction for TXT/PDF/IMG/DOCX and OCR	Implemented
FR-04	Retrieval pipeline: BM25 + embedding rerank + answer guard	Implemented
FR-05	Open file by exact indexed file name	Implemented
FR-06	Wake word + STT + TTS with fallbacks	Implemented
FR-07	Floating bubble + timer bubble integration	Implemented
FR-08	Automation rules: when X, do Y (offline)	Implemented
FR-09	Local personalization memory with explicit consent	Implemented
FR-10	Multimodal capture and OCR follow-up Q/A	Implemented
FR-11	Online mode toggle with policy gating	Implemented

2.3 Non-Functional Requirements

- NFR-01 Privacy: offline default, policy-gated network access, no fake cloud claims.
- NFR-02 Reliability: robust fallback chain for Android file read failures.
- NFR-03 Performance: responsiveness under low-memory profile with throttled maintenance.
- NFR-04 Scalability: full-scan content search with candidate ranking and cache invalidation.
- NFR-05 Security: endpoint allowlisting and encrypted local personalization storage.
- NFR-06 Maintainability: modular source layout, regression test gates, signed release flow.

2.4 Platform and Constraints

- Primary runtime: Capacitor Android app and WebView environment.
- Android-specific constraints: scoped storage, special app access permissions, OEM differences.
- Resource limits: low-end devices require adaptive profile selection.
- Special permissions needed for full feature set: `MANAGE_EXTERNAL_STORAGE` and `SYSTEM_ALERT_WINDOW`.

3) Complete Implementation Plans

3.1 Original End-to-End Build Plan (Phase 1 to Phase 5)

- Phase 1 Foundation: Vite setup, Capacitor integration, UI skeleton, permission architecture.
- Phase 2 Core Logic: parser-first development, intent routing, speech integration, native launch actions.
- Phase 3 Offline AI + RAG: model bundling, worker threading, vector storage and retrieval hooks.
- Phase 4 Android Capabilities: recursive file scanner, OCR/PDF extraction, service/bubble support.
- Phase 5 Test + Release: profiling, edge-case test expansion, signed APK generation.

3.2 v1 Hardening Plan Executed

- Central runtime policy module: offline_default true, online session opt-in gating.
- Retrieval upgrade: BM25 stage + embedding rerank stage, content-first ordering.
- Mic arbitration and wake-word/STT reliability hardening with fallback behavior.
- Android stabilization: permission/bubble states and fallback-safe storage strategy.
- Release gate: required model assets + tests + production build verification.
- Phase-2 hook planning for local LLM route (deterministic intents remain first-class).

3.3 Extended Feature Integration Plan

- Knowledge graph with citations + confidence scoring.
- Incremental index maintenance and change detection.
- Action planner for safe multi-step task execution.
- Encrypted personalization memory (user consent driven).
- Context-aware proactive suggestions from local behavior and indexed context.
- Performance governor with battery/data-saver aware scheduling.

4) System Architecture and Design

4.1 Module Map

Module	Primary File(s)	Design Responsibility
Command Engine	src/assistant.js	Intent parsing, response routing, policy-aware execution
Search/RAG	src/search-engine.js	File reading, extraction, scoring, rerank, answer synthesis
Filesystem	src/filesys.js	Indexing, file metadata, open/search helpers, Android fallback paths
Runtime Policy	src/policy.js	Offline default, online session gating, feature allow checks
Voice Layer	src/main.js + workers	Wake word/STT/TTS orchestration, mic state handling
Automation	src/automation-engine.js	Offline trigger-action rule persistence and execution
Knowledge	src/knowledge-graph.js	Citation memory and confidence-aware fallback context
Personalization	src/usage-learning.js + src/secure-store.js	Encrypted local learning profile with user consent
Performance	src/performance-governor.js	Adaptive profile by RAM/CPU/battery/network saver

4.2 Storage Design

- IndexedDB: file index + embedding cache + secure key store (for encrypted personalization).
- localStorage: lightweight settings/state (policy defaults, toggles, timestamps).
- In-memory caches: query cache and analysis cache with TTL and explicit invalidation paths.

5) Data Flow Diagrams and Design Flows

5.1 High-Level Architecture (Mermaid)

```
graph TD
    User([User Voice/Text]) --> UI[Frontend Chat Interface]
    UI --> IntentParser{Intent Parser}

    IntentParser -- "Device Actions" --> NativeBridge[Capacitor Native Bridge]
    NativeBridge --> OS[Android OS Services]

    IntentParser -- "Knowledge Queries" --> RAG[RAG Search Engine]
    RAG --> FileSys[Local Storage Scanner]
    FileSys --> Index[(IndexedDB Index + Meta)]

    RAG --> EmbedWorker[[Embedding Worker]]
    EmbedWorker --> BM25[BM25 Stage A]
    EmbedWorker --> Rerank[Embedding Rerank Stage B]
    Rerank --> Guard[Answer Guard + Confidence]
    Guard --> KG[(Knowledge Graph)]

    Guard --> UI
    UI --> TTS[Text-to-Speech]
    TTS --> User
```

5.2 File Indexing and Retrieval Sequence

```
sequenceDiagram
    participant U as User
    participant SE as Search Engine
    participant FS as File System
    participant EXT as Extractors (PDF/OCR/DOCX)
    participant IDX as IndexedDB

    Note over FS,IDX: Background Indexing
    FS->>FS: Scan storage roots recursively
    FS->>SE: Send candidate file entries
    SE->>EXT: Read/parse per file type
    EXT-->>SE: Return extracted text
    SE->>IDX: Save metadata/cache keys

    Note over U,SE: Query-time Retrieval
    U->>SE: Ask question
    SE->>IDX: Load readable files
    SE->>SE: Stage A BM25 candidate scoring
    SE->>SE: Stage B embedding rerank (worker)
    SE->>SE: Confidence guard + context synthesis
    SE-->>U: Answer + evidence + confidence
```

5.3 Android Read Fallback Flow

```
flowchart TD
    Start([Need file bytes]) --> ReadFile[Filesystem.readFile]
    ReadFile -- Success --> Parse[Extract text]
    ReadFile -- Failure --> URI[Filesystem.getUri]
    URI --> Convert[Capacitor.convertFileSrc]
    Convert --> Fetch[fetch(webElement)]
    Fetch -- Success --> Parse
    Fetch -- Failure --> FilenameFallback[Filename match fallback]
    FilenameFallback --> Return[Return partial result]
    Parse --> Return
```

5.4 Runtime Policy and Online Gating

```
flowchart TD
    Request([Network Feature Request]) --> Check1{offlineDefault=true}
    Check1 --> Check2{onlineSessionOptIn=true?}
    Check2 -- No --> Block[Return policy-blocked message]
    Check2 -- Yes --> Check3{Feature allowed?}
    Check3 -- No --> Block
    Check3 -- Yes --> Allow[Allow guarded fetch to allowlisted endpoint]
```

5.5 Voice/Mic Arbitration Flow

stateDiagram-v2

```
[*] --> IDLE
IDLE --> WAKE_WORD: Wake-word enabled
WAKE_WORD --> LISTENING: Trigger phrase detected
LISTENING --> PROCESSING: STT result captured
PROCESSING --> IDLE: Response delivered
LISTENING --> IDLE: Cancel/timeout/error
WAKE_WORD --> IDLE: Wake-word disabled
```

5.6 Build/Release Pipeline

flowchart LR

```
Code[Source + Config] --> Tests[Run release:check]
Tests --> Build[Build web assets]
Build --> Sync[npx cap sync android]
Sync --> Sign[Gradle assembleRelease with keystore]
Sign --> Verify[apksigner verify + hash]
Verify --> APK[Release APK output]
```

6) Testing, Validation, and Quality Gates

- Unit and behavior tests cover parser, policy, retrieval utils, search hardening, worker lifecycle, and stability forecast.
- Release gate enforces: required local model assets + all test suites + production build success.
- Android deployment validation includes install, launch, process checks, and runtime smoke interactions.
- APK signing verification ensures installable release integrity.

Current Regression Test Families

- tests/test-all.mjs
- tests/policy-runtime.test.mjs
- tests/retrieval-utils.test.mjs
- tests/search-engine-hardening.test.mjs
- tests/offline-brain-hardening.test.mjs
- tests/embedding-client.test.mjs
- tests/next-features-hardening.test.mjs
- tests/stability-forecast.test.mjs

7) Future Risk Forecast and Mitigation Design

- R-01 Large-file OCR/parse latency: Adaptive throttling, worker offload, staged preview updates.
- R-02 Android OEM permission variance: Fallback UX with explicit state labels and settings deep links.
- R-03 Storage rescans missing some locations: Multi-root scan strategy + incremental maintenance + stale-index refresh.
- R-04 Wrong-answer risk on weak evidence: Answer guard, overlap checks, confidence threshold, clarification prompt.
- R-05 Voice conflicts wake-word vs STT: Mic arbiter state machine with pause/resume contract.
- R-06 Performance on low-end hardware: Performance governor with battery/data-saver aware profile downgrades.
- R-07 Feature drift from duplicate logic: Regression tests and dead-path removal in main flows.

8) Release and Distribution Design

- Release channel: signed sideload APK for Android deployment.
- Build output location: F:/Hackathon Project/release
- Installation path: adb install -r
- Post-install validation: launch app, check permissions, scan files, search content, test voice and bubble.

Appendix A) Original Full Implementation Plan (verbatim source)

Project Implementation Plan: "Nice" Offline AI Assistant

1. Project Overview & Description

"Nice" is a privacy-first, fully offline AI Assistant application designed primarily for Android devices (via Capacitor) and desktop web browsers. Unlike traditional AI assistants that send all user data to the cloud, Nice runs large language models (LLMs), Wake Word detection, Vector Embeddings, and Optical Character Recognition (OCR) directly on the device's edge hardware using WebGPU and WebAssembly.

Its primary goal is to provide a fully transparent, highly responsive, and data-secure conversational interface capable of interacting with the device's native ecosystem (files, calendar, apps, alarms) without an internet connection.

2. Core Functionalities

The application is divided into several modules of functionality:

2.1 Natural Language Processing & Local AI

- ****On-Device LLM Inference:**** Uses ONNX Runtime Web and WebGPU to run a local language model for chatting and understanding context.
- ****Wake Word Detection:**** Continuously listens for a specific trigger phrase (e.g., "Hey Nice") using an efficient, lightweight local audio-classification model.
- ****RAG (Retrieval-Augmented Generation):**** Embeds local documents and knowledge using Transformer-based feature extraction (`@xenova/transformers`) and stores them in a local Vector Database (IndexedDB) for answering context-specific questions.

2.2 Device & OS Integration

- ****Deep File Search (Native):**** Recursively indexes the entire Android external storage, supporting over 50 file types (PDF, Word, TXT, Excel, Code). Extracts text and OCRs images.
- ****Intent Parsing System:**** A regex and rule-based fallback system that can route immediate commands:
 - App Launching (e.g., "Open Spotify")
 - Alarm & Calendar Management
 - Media Controls ("Play Bohemian Rhapsody")
 - Weather parsing and mathematical expression evaluation (including "word math" like "two times four").

2.3 Audio & Accessibility

- ****Speech-to-Text (STT):**** Local transcription or Whisper-based models.
- ****Text-to-Speech (TTS):**** Uses the native Android TTS engine or Web Speech API to read responses aloud.
- ****Floating Chat Head:**** An overlay service (Service Window) that allows the assistant to be summoned from anywhere in the OS.

3. Technology Stack

Category	Technology	Purpose
Frontend UI/UX	Vanilla HTML/CSS/JS, Vite	High performance, lightweight DOM manipulation without massive framework overhead.
Mobile Runtime	Capacitor (v8)	Wraps the web app into a native Android APK, providing bridges to native device features.
Local AI Engine	ONNX Runtime Web, `@xenova/transformers`	Runs AI models (LLMs, Whisper, Embeddings) locally using Wasm/WebGPU.
Text Extraction	`pdfjs-dist`, `tesseract.js`, `mammoth`	Extracts raw text from PDFs, Images (OCR), and Word Documents respectively.
Database	IndexedDB	Local storage of file metadata, text chunks, and vector embeddings for RAG.

4. Software Development Lifecycle (Step-by-Step Approach)

Building this from scratch follows a 5-phase approach.

Phase 1: Foundation & Project Setup

- Initialize Project:** Create Vite vanilla JS template.
- Capacitor Integration:** Install `@capacitor/core`, `@capacitor/android`, and initialize the [capacitor.config.json](file:///f:/Hackathon%20Project/capacitor.config.json).
- UI Prototyping:** Build the chat interface, microphone button, and settings menu. Implement CSS variables for dark/light themes.
- Permissions Architecture:** Implement a startup sequence that requests all necessary Android permissions (`RECORD\_AUDIO`, `MANAGE\_EXTERNAL\_STORAGE`, `SYSTEM\_ALERT\_WINDOW`).

Phase 2: Core Assistant Logic & Intents

- Command Parser:** Build [test-all.mjs](file:///f:/Hackathon%20Project/tests/test-all.mjs) first (TDD approach). Implement Regex-based extraction modules for Alarms, Calendars, App Launching, and Math.
- Capacitor Plugins Integration:** Use the `@capacitor/app-launcher` to trigger Android intents for opening apps and setting alarms via standard URIs.
- Speech Integration:** Wire up the `window.SpeechRecognition` (or polyfill) for listening to the user, and `window.speechSynthesis` for the assistant's voice.

Phase 3: The Offline AI & RAG Engine

- Model Bundling:** Create scripts ([download-model.mjs](file:///f:/Hackathon%20Project/download-model.mjs), [download-tess.mjs](file:///f:/Hackathon%20Project/download-tess.mjs)) to pull required ONNX models locally into the `public/` directory so the APK ships with them.
- Web Worker Threading:** Build [worker.js](file:///f:/Hackathon%20Project/src/clip-worker.js) to offload the heavy WebGPU LLM inference and embedding calculations so the main UI thread never freezes.
- Vector Database:** Build a minimalist class wrapping IndexedDB to store 384-dimensional arrays (embeddings) and do Cosine Similarity matching for searches.

Phase 4: Native Android Capabilities & Indexing

- File System Scanner** ([filesystem.js](file:///f:/Hackathon%20Project/src/filesys.js)): Implement the recursive scanner utilizing `@capacitor/filesystem`. Handle Android 11+ Scoped Storage by falling back to `FileSystem.getUri` + `fetch` for reading file bytes.
- Text Extraction Pipeline** ([search-engine.js](file:///f:/Hackathon%20Project/src/search-engine.js)): Integrate Tesseract for `.png/.jpg`, PDF.js for `.pdf`. Slice extracted text into paragraphs.
- Background Services:** Implement the Android `Service` wrapper in Java to support the floating chat head overlay, keeping the WebView alive in the background.

Phase 5: Testing, Optimization, and Release

- Performance Profiling:** Optimize memory limits. Throttle the file scanner (e.g., yielding every 30ms) to avoid "App Not Responding" (ANR) crashes on low-end phones.
- Testing suite:** Expand the TDD suite to cover edge cases like division by zero in word math, or missing file extensions.
- APK Build:** Run Android Studio/Gradle to generate the signed `release.apk` mapping `targetSdkVersion 34`.

5. Data Flow & Architecture Diagrams

5.1 High-Level Architecture

```

graph TD
    User([User Voice/Text]) --> UI[Frontend Chat Interface]
    UI --> IntentParser[Intent Parser]

    IntentParser -- "Launch App, Alarm, Math" --> NativeBridge[Capacitor Native Bridge]
    NativeBridge --> OS[Android OS Services]
  
```

```

IntentParser -- "General Query" --> RAG[RAG & Search Engine]
IntentParser -- "File Search" --> RAG

RAG --> FileSys[Local Storage Scanner]
FileSys --> VectorDB[(IndexedDB Vectors & Meta)]

RAG --> AIWorker[[AI Web Worker Thread]]
AIWorker --> ONNX[ONNX WebGPU Runtime]
ONNX --> LLM[(Local LLM Model)]

ONNX --> ResponseGen[Generate Response]
ResponseGen --> UI
UI --> TTS[Text-to-Speech Engine]
TTS --> User
...

```

5.2 File Indexing & Search RAG Data Flow

```

```mermaid
sequenceDiagram
 participant U as User
 participant SE as Search Engine
 participant FS as File System
 participant EXT as Extractors (PDF/OCR)
 participant VDB as Vector DB / Index
 participant LLM as Local LLM

 %% Indexing Phase
 Note over SE, VDB: 1. Background Indexing Phase
 FS->>FS: Scan entire Android Root Storage
 FS->>SE: Yield File List (Text, PDFs, Images)
 SE->>EXT: Send raw bytes for unsupported formats
 EXT-->>SE: Return raw text strings
 SE->>SE: Chunk text into paragraphs
 SE->>VDB: Store Metadata + Text Chunks

 %% Retrieval Phase
 Note over U, LLM: 2. Active Search & Retrieval
 U->>SE: "What is my March invoice total?"
 SE->>SE: Tokenize Query ("March", "invoice", "total")
 SE->>VDB: Query matching paragraphs (BM25 or Cosine)
 VDB-->>SE: Return top 5 relevant chunks
 SE->>LLM: Prompt Context: [Chunks] + "Answer the user."
 LLM-->>U: "Your March invoice total is $3,847.50."
...

```

### ### 5.3 Error Handling & Fallback Strategy (Android 11+)

```

```mermaid
flowchart TD
    Start[Request File Read] --> CapRead[Try Filesystem.readFile]
    CapRead -- Success --> Parse[Extract Text]
    CapRead -- Failure --> GetURI[Try Filesystem.getUri]
    GetURI -- Return URI --> Convert[Capacitor.convertFileSrc]
    Convert --> Fetch[Try fetch Web URL]
    Fetch -- Success --> Parse
    Fetch -- Failure --> NameFallback[Match by Filename Only]
    NameFallback --> Display[Return Partial Search Result]
    Parse --> Display
...

```

Summary

Building "Nice" requires a careful balance between modern Web AI (ONNX, Transformers.js) and restrictive Android Native APIs (Scoped Storage, Background Services). By keeping the ML inference inside Web Workers and building robust error catchers/fallbacks for the file system bridge, the application achieves a highly responsive, entirely private assistant experience.

Appendix B) Original PRD Source (verbatim)

Product Requirements Document: Nice Offline Mobile Assistant

****Version****: 1.0.0
****Status****: Draft
****Owner****: AI Assistant Team

1. Executive Summary

The **Nice Offline Mobile Assistant** is a privacy-first, high-performance personal assistant designed to operate entirely on-device. By leveraging Small Language Models (SLMs) and modern WebAssembly-based AI, it eliminates the need for cloud connectivity, ensuring data remains private and the assistant remains functional even in "off-grid" scenarios. It bridges the gap between web-based AI and native mobile automation.

2. Target Audience

- ****Privacy Enthusiasts****: Users who do not want their data sent to cloud servers for processing.
- ****Low-Connectivity Users****: Individuals in areas with unreliable internet who still need intelligent assistance.
- ****Power Users****: Users seeking fast, local file search and device automation without cloud latency.
- ****Security-Conscious Organizations****: Entities requiring offline AI for confidential data handling.

3. User Stories

- ***As a user in a remote area***, I want to ask my assistant for information from my local PDFs without an internet connection.
- ***As a privacy-focused user***, I want to use voice commands to set alarms and play music without my voice being recorded on a server.
- ***As a student***, I want to quickly search through hundreds of textbook pages for a specific concept using natural language.
- ***As a professional***, I want to take voice notes and have them transcribed instantly while traveling.

4. Functional Requirements

4.1 Voice Interaction & AI

- ****Wake Word Detection****: Identify a specific trigger word to activate the assistant.
- ****Offline speech-to-Text (STT)****: Transcribe user speech using `@xenova/transformers` (Whisper) completely offline.
- ****Text-to-Speech (TTS)****: Provide spoken responses using native system voices.
- ****Local SLM****: Process natural language queries using `@mlc-ai/web-llm` (WebLLM) on the device's CPU/GPU.
- ****Contextual Memory****: Maintain a short-term chat history for follow-up questions.

4.2 Intelligence & File Parsing

- ****Deep-File Search****: Index and search local PDF, Docx, and Text files.
- ****Image OCR****: Extract text from images and screenshots using `Tesseract.js`.
- ****Sliding Window Scoring****: Use advanced ranking (BM25-lite) to find precise "pinpoint" answers within large documents.
- ****File Watcher****: Automatically detect and index new or modified files in specific directories.

4.3 Device Automation

- ****App Launcher****: Open installed Android applications (Spotify, Maps, Calculator, etc.).
- ****Media Control****: Play, pause, and search for local audio files.
- ****Alarms & Timers****: Set, cancel, and manage system-level alarms and countdown timers.
- ****Calendar & Flashlight****: Native integration for scheduling and hardware controls (vibration, flashlight).

4.4 Utility Features

- ****Smart Math****: Handle complex math/percentage calculations via natural language (e.g., "What is 15% of 850?").
- ****Local Notes****: Create, view, and delete notes persisted in `localStorage`.
- ****Battery & System Info****: Report current device status (battery level, storage stats).

5. Non-Functional Requirements

- ****Privacy****: Zero data transmission to external servers.
- ****Performance****:
 - Search response time < 100ms.
 - AI inference time optimized for mobile hardware (WebGPU/WASM).
- ****Efficiency****: Memory management logic to prevent crashes on 4GB RAM devices (5MB hardware cap for large file chunks).
- ****Offline-First****: All core models (Whisper, LLM) must be bundle-able or downloadable for permanent offline use.

6. Technical Architecture

- ****Frontend****: Vite + Vanilla JavaScript + CSS (Glassmorphism design).
- ****Native Bridge****: Capacitor (coupling web views with Android hardware APIs).

- **AI Engine**:
 - WebLLM (MLC-AI) for the Language Model.
 - Transformers.js (ONNX) for Whisper STT.
- **Data Layer**:
 - **IndexedDB**: Persistent file index and metadata.
 - **LocalStorage**: User settings, chat history, and notes.
 - **Filesystem API**: Native read/write access via `capacitor-file-system`.

7. UI/UX Vision

- **Modern Aesthetic**: Dark mode support, smooth CSS animations, and a "vibrant" premium feel.
- **Accessibility**: Clear visual indicators for listening/processing states.
- **Responsiveness**: Mobile-first design that scales to tablets and desktops.

8. Future Roadmap

- **Vision Integration**: Local image classification and object detection.
- **Plugin System**: Allow users to add custom automation scripts.
- **Improved RAG**: Multi-modal search (searching images by content descriptions).
- **Smart Home Integration**: Local control for IoT devices (Matter/Thread support).